

#27 Coding Tips - 2

15/06/2025

Blank lines to organise code

bad

```
name = input("Enter your name here: ").strip()
```

```
while not name:
```

```
    print("That's an invalid name. Try again:")
```

```
    name = input().strip()
```

```
print(f"Welcome {name}!")
```

```
print("What would you like to do?")
```

better

```
name = input("Enter your name here: ").strip()
```

```
while not name:
```

```
    print("That's an invalid name. Try again: ")
```

```
    name = input().strip()
```

```
print(f"Welcome {name}!")
```

```
print("What would you like to do?")
```

Divided into three parts:

- initial user input

- input validation

- using the variable

15/06/2025

Name functions appropriately

- Choose descriptive function names. For instance using `display_` or `print_` when the function's purpose is to output values.

You shouldn't have to look at a function's implementation to know what it does.

- Ensure a function does one thing and is named appropriately. A function named `total` should return a value. `print_total` should display something and return `None`.

Don't mutate the collection during iteration

- Suppose we have a list of strings and we want to iterate over that list and print out each element.

```
1 words = ['scooby', 'do', 'on', 'channel', 'two']
2
3 for word in words:
4     print(word) # prints: scooby, on, two (in that order)
5     words.remove(word)
6
7 print(words) # prints: ['do', 'channel']
```

- The `list.remove` method removes the element passed to it as the argument from the list.

We expect all elements to be removed by the end of the iteration. However, that is not the case.

15/06/2025

What's happening?

- In the first iteration, `word` points to the first element of `words`, namely, `'scooby'`. That word gets printed and it is then removed from `words`.
- Now, `words` contains `['do', 'on', 'channel', 'two']`
- In the second iteration, `word` now points to the second element of `words`, which is now `'on'`. Thus, at this point, we print `'on'`. When `words.remove()` is called, the element `'on'` is removed. The element `'do'` has been skipped.

Why?

The `for` loop iterates index-by-index. As `'do'` during the second iteration was at index 0 and the loop had already accessed element 0 at the first iteration prior to removing `'scooby'`. Next element accessed was at index 1, which is `'on'`.

- Now left with `words = ['do', 'channel', 'two']`
- In the third iteration, `word` now points to element at index 2, which is now `'two'`. Thus, `'two'` is printed and then removed.
- `words` now contains `['do', 'channel']`
- Python attempts fourth iteration. However, there are only two elements remaining and hence no element at index 3 as there is only index 0 and index 1.

CONSIDERING THE ABOVE EXAMPLE, DO NOT MUTATE A COLLECTION WHILE ITERATING THROUGH IT.

OR AT TO MUTATE INDIVIDUAL ELEMENTS, NOT THE COLLECTION ITSELF

15/06/2025

this list is an
element

```
pairs = [[6, 'scobis'], [2, 'do'], [2, 'on'], [7, 'channel'],  
         [3, 'two']
```

```
for pair in pairs:  
    pair.pop()
```

```
print(pairs) # prints: ['scobis', 'do', 'on', 'channel', 'two']
```

Variable shadowing

Variable shadowing occurs when the local variable (in a function for example) shares the same name as the variable in the outer scope.

Not good practice as it can lead to bugs and confusion as it prevents access to the outer scope variable from within the inner scope.

```
name = 'johnson'
```

```
def print_full_name():  
    for name in ['kim', 'joe', 'sam']:  
        print(f'{name} {name}')
```

```
print_full_name()
```

```
# kim kim
```

```
# joe joe
```

```
# sam sam
```

```
print(name) # 'johnson'
```


15/06/2025

In this example, the outer scoped variable `name` has been shadowed - the variable `name` in the `for` loop represents the elements in the list `['kim', 'joe', 'sam']`.

The following is an example of reassignment:

```
name = 'Johnson'
```

```
def print_full_name():  
    global name  
    for name in ['kim', 'joe', 'sam']:  
        print(f"{name} {name}")
```

```
print_full_name()
```

```
# Prints:
```

```
# kim kim
```

```
# joe joe
```

```
# sam sam
```

```
print(name) # sam
```

When printing `name` on the last line, the value is now `'sam'`.

During the `for` loop, during each iteration, the respective `name` in `['kim', 'joe', 'sam']` is assigned to the global `name`. As `'sam'` is the last name to be assigned, the global variable `name` now has `'sam'` assigned to it for when the last line `print(name)` is executed.

The best approach would be to rename the `for` loop variable to something more precise like `first_name`.

15/06/2025

```
name = 'johnson'
```

```
def print_full_name():
    for first_name in ['kim', 'joe', 'sam']:
        print(f"{first_name} {name}")
```

```
print_full_name()
```

```
# Prints:
```

```
# kim johnson
```

```
# joe johnson
```

```
# sam johnson
```

```
print(name) # johnson
```

BE VERY CAUTIOUS ABOUT SELECTING APPROPRIATE VARIABLE NAMES, ESPECIALLY WHEN USING LOOPS OR NESTED FUNCTIONS

Use underscore for unused identifiers

16/06/2025

- When you have a list of names, you want to print out a string for every name in the list, but you do not care about the actual names. Use an underscore:

```
names = ['kim', 'joe', 'sam']
for _ in names:
    print('Got a name!')
```

Another example is when you need the first item (typically) the index of a sequence in a loop but do not need

16/06/2025

the second item (the value). You can use `-` to indicate that the second item is not being used in the loop.

```
names = ['kim', 'joe', 'sam']  
for index, _ in enumerate(names):  
    print(f"{index}: Got a name!")
```

tuple unpacking

prints

0: Got a name!

1: Got a name!

2: Got a name!

NB: The `enumerate` function is used to iterate over a sequence in a way that makes both the index and the value available to the loop body.

Thus, the above loop iterates over `names`, with each iteration passing the index to the `index` variable, and the element to the `_` variable.

CHAT GPT

- `enumerate(names)` takes an iterable (like a list) and returns an enumerator object.
- This object produces pairs of values:
(`<index>`, `<item>`) for each element in the list.



(0, 'kim')

(1, 'joe')

(2, 'sam')

7/6/2025

Can experience through struggling

Beginners often feel:

1. they must know the "best" or "right" way to do something so they feel they must learn "best practice".
2. that they waste too much time debugging and not doing things correctly the first time.

For 1. it is best to learn why something is "best practice" and that is through learning/implementing other ways and making mistakes.

For 2. it is not bad to struggle through the journey of the "bad" or sub-optimal practices first. That is getting experience.

DON'T MEMORISE "BEST PRACTICES", INSTEAD SPEND ENOUGH TIME PROGRAMMING TO THE POINT WHERE YOU UNDERSTAND THE CONTEXT FOR THOSE PRACTICES.

SPEND TIME PROGRAMMING

SPEND TIME DEBUGGING

STRUCTURE

SEARCH FOR THE RIGHT TERMS

PLAY AROUND WITH THE CODE